Strings

28 December 2024 16:51

Thin python strings are one of the most widely used data types and are fundamental for handling text data. A string is a sequence of characters enclosed within either single quotes('), double quotes ("), or triple Quotes(''' or """). Strings in python are immutable, meaning they cannot be changed after creation.

```
>>> name = 'Shiv'
>>> name
'Shiv'
>>> greet = "Hello,"
>>> greet
'Hello,'
>>> type(greet)
<class 'str'>
>>> multi_line = ''' This is
... a multi-line
... string.
... '''
>>> multi_line
' This is \na multi-line \nstring.\n'
>>>
```

newline character automatically inserted by Python's interpretor.

```
>>> multi_line
' This is \na multi-line \nstring.\n'
>>> print(multi_line)
 This is
a multi-line
string.
```

← when print() function is used, it displays the string in multiple lines.

Accessing Strings

indexing

	0	1	2	3
name	S	h	i	v
	-4	-3	-2	-1

```
>>> name = 'Shiv'
>>> name
'Shiv'
>>> greet = "Hello,"
>>> greet
'Hello,'
>>> type(greet)
<class 'str'>
>>> multi_line = ''' This is
... a multi-line
... string.
... '''
>>> multi_line
' This is \na multi-line \nstring.\n'
>>>
```

text

-3 -2 -1

Why -ve indexing?

name[0] ← It represents the 1st element

name[-1] ← " " " last element.

```
>>> name
'Shiv'
>>> name[0]
'S'
>>> name[-1]
'v'
>>> name[8]
```

invalid index

```
's'
>>> name[-1]
'v'
>>> name[8]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

invalid index

Slicing

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
P	y	t	h	o	n		P	r	o	g	r	a	m	m	i	n	g
-18	-17	-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Syntax:

str_var[start : stop : step]

optional

inclusive

exclusive (stop - 1)

the indexing always moves forward direction

text [-12 : -5]

text [-5 : 12] ← it returns '' empty

text [-12 : 5]

If step value is -ve then picking starts from back position

```
>>> text = 'python programming'
>>> text[::]
'python programming'
>>> text[::2]
'pto rgamn'
>>> text[1::2]
'yhnpormig'
>>> text[-12:-5]
' progra'
>>> text
'python programming'
>>> text[-5:-12]
''
>>> text[-5:-12:-1]
'margorp'
```

← characters picked from end to front as step value is -ve.

Experiment to reverse the string

```
>>> text
'python programming'
>>> text[-1:0:-1]
'gnimmargorp nohty'
>>> text[-1::-1]
'gnimmargorp nohtyp'
>>> text[::-1]
'gnimmargorp nohtyp'
```

Home work:

Write a code to check a word is palindrome or not? Print appropriate messages.

Concatenation: Combine two or more strings using the plus(+) operator.

Example:

```
>>> friname = 'nitin'
```

```
>>> friname[::-1]
'nitin'
>>> greet
'Hello,'
>>> print(greet+' '+friname)
Hello, nitin
```

Repetition: Repeat a string multiple times using * operator.

Example:

```
>>> print('*'*60)
************************************************************
>>> print('-'*60)
------------------------------------------------------------
>>> print('-'*100)
----------------------------------------------------------------------------------------------------
>>> print('Jiya'*100)
JiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiy
aJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJi
yaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJ
iyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiya
JiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiyaJiya
```

String Methods

Python provides a rich set of built in methods to manipulate strings.

Case Conversion

- **'lower**()': Converts to lowercase.
- **'upper()':** Converts to uppercase.
- **'capitalize**()': capitalize the first character.
- **'title**(): Capitalizes the first character of each word.
- **'swapcase**()': Swaps the case of each character.

Examples:

```
>>> text
'python programming'
>>> t1 = text.upper()
>>> t1
'PYTHON PROGRAMMING'
>>> t2 = t1.lower()
>>> t2
'python programming'
>>> t3 = t2.capitalize()
>>> t3
'Python programming'
>>> t4 = t2.title()
>>> t4
'Python Programming'
>>> t5 = t4.swapcase()
>>> t5
'pYTHON pROGRAMMING'
>>>
```

Search and Replace

- **'find()':** Finds the first occurrence of a substring. Returns -1 if not found.
- **'index()':** Similar to find() but raises an exception if not found.
- **'count()':** Counts occurrences of a substring.
- **'replace()':** Replaces occurrences of a substring with another string. And it returns a new string.

Examples:

```
>>> text
'python programming'
>>> text.find('o')
4
>>> text.find('i')
15
>>> text.find('u')
-1
>>> text.index('o')
4
>>> text.index('i')
15
>>> text.index('u')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: substring not found
>>> text.count('p')
2
>>> text.count('t')
1
>>> text.count('o')
2
>>> text.count('u')
0
>>>
```

```
>>> text
'python programming'
>>> t1 = text.replace('python', 'C++')
>>> t1
'C++ programming'
>>> t1 = text.replace('p', 'C++')
>>> t1
'C++ython C++rogramming'
>>> t1 = text.replace('u', 'C++')
>>> t1
'python programming'
>>>
```

substring must be present in the string to be replaced.

- **P2.1** Write a program that displays the dimensions of a letter-size (8.5 × 11 inch) sheet of paper in millimeters. There are 25.4 millimeters per inch. Use constants and comments in your program.
- **P2.2** Write a program that computes and displays the perimeter of a letter-size (8.5 × 11 inch) sheet of paper and the length of its diagonal.
- **P2.3** Write a program that reads a number and displays the square, cube, and fourth power. Use the `**` operator only for the fourth power.
- ▪ **P2.4** Write a program that prompts the user for two integers and then prints
  - The sum
  - The difference
  - The product
  - The average
  - The distance (absolute value of the difference)
  - The maximum (the larger of the two)
  - The minimum (the smaller of the two)

  Hint: Python defines `max` and `min` functions that accept a sequence of values, each separated with a comma.

String Validation

- **'isalpha()'**: Returns True if all characters are alphabetic.
- **'isdigit()'**: Returns True if all characters are digits.
- **'isalnum()'**: Returns True if all characters are alpha numeric.
- **'isspace()'**: Returns True if the string contains only white space.

For example:

```
>>> text = "Hello123"
>>> text.isalpha()
False
>>> text.isdigit()
False
>>> text.isalnum()
True
>>> text.isspace()
False
```

String formatting

1. Using % operator

```
>>> name = "Jiya"
>>> age = 12
>>> print('Name: %s, Age: %d'%(name,age))
Name: Jiya, Age: 12
>>> salary = 5000.45633
>>> print('Salary: %.2f'%(salary))
Salary: 5000.46
```

2. Using str.format()

https://www.w3schools.com/python/ref_string_format.asp
For more click the above link.

Examples:

```
>>> name
'Jiya'
>>> age
12
>>> print("Name: {}, Age: {}".format(name, age))
Name: Jiya, Age: 12
>>> print("Salary: {}".format(salary))
Salary: 5000.45633

>>> print("Salary: {:.2f}".format(salary))
Salary: 5000.46
```

3. Using f-string(introduced in Python 3.6 version)

```
>>> name
'Jiya'
>>> age = 12
>>> salary
5000.45633
>>> print(f'Name: {name}, Age: {age}')
Name: Jiya, Age: 12
>>> print(f'Name: {name}, Age: {age}, Salary: {salary:.2f}')
```

```
Name: Jiya, Age: 12, Salary: 5000.46
```

String Immutability

Strings in python are immutable, meaning once a string is created, it cannot be changed.

```
>>> name
'Jiya'
>>> name[0]
'J'
>>> name[0] = 'H'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

```
>>> name1 = 'H'+name[1:]
>>> name
'Jiya'
>>> name1
'Hiya'
```

65+26−1 = 90

A ← 65	a ← 97	0 ← 48
B ← 66	⋮	⋮
⋮	⋮	⋮
Z ← 90	z ← 122	9 ← 57

spacebar = 'a' − 'A' = 32.

2 byte = 16 bits

2^{16}

2^16=65,536

Symobls

28 states.

Common Built-in Functions

- **'len()'**: The returns the length of the string.
- **'max()'**: Returns the character with the highest Unicode value.
- **'min()'**: Returns the character with the lowest Unicode value.
- **'sorted()'**: Returns a sorted list of characters.

About Unicode methods in Python:

```
>>> ord('A')
65
>>> ord('B')
66
>>> chr(66)
'B'
```

Examples:

```
>>> name
'Jiya'
>>> len(name)
4
>>> max(name)
'y'
>>> min(name)
'J'
>>> sorted(name)
['J', 'a', 'i', 'y']
```

Escape Sequences

Special characters can be used in strings with a backslash('\').

Escape Sequence	Description
\n	Newline
\t	Tab
\'	Single quote
\"	Double quote
\\	Backslash

```
>>> escape = '\'Hello!\''
>>> print(escape)
'Hello!'
```

Raw Strings

Prefixing a string with 'r' or capital 'R', treats it as a raw string and ignores escape sequences.

```
>>> path = "C:\Users\App"
  File "<stdin>", line 1
    path = "C:\Users\App"
           ^^^^^^^^^^^^^^
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position
2-3: truncated \UXXXXXXXX escape
>>> path = r"C:\Users\App"
>>> path
'C:\\Users\\App'
>>> print(path)
C:\Users\App
```

← raw string to avoid error generated by Python.

← stored in memory

← processed output by print() function.

Summary of Strings in Python

- **String are immutable sequences of characters**
- **They support indexing, slicing and a rich set of methods.**
- **Formatting and manipulation are straightforward using methods like format(), or f-strings.**
- **Escape sequence and raw strings handles special characters effectively.**

Home work: 31/12/24

▪▪ **P2.6** Write a program that prompts the user for a measurement in meters and then converts it to miles, feet, and inches.

▪ **P2.7** Write a program that prompts the user for a radius and then prints
- The area and circumference of a circle with that radius
- The volume and surface area of a sphere with that radius

▪▪ **P2.8** Write a program that asks the user for the lengths of the sides of a rectangle. Then print
- The area and perimeter of the rectangle
- The length of the diagonal

▪ **P2.9** Improve the program discussed in How To 2.1 to allow input of quarters in addition to bills.

▪▪▪ **P2.10** Write a program that helps a person decide whether to buy a hybrid car. Your program's inputs should be:
- The cost of a new car
- The estimated miles driven per year
- The estimated gas price
- The efficiency in miles per gallon
- The estimated resale value after 5 years

© asiseit/iStockphoto.

Compute the total cost of owning the car for five years. (For simplicity, we will not take the cost of financing into account.) Obtain realistic prices for a new and used hybrid and a comparable car from the Web. Run your program twice, using today's gas price and 15,000 miles per year. Include pseudocode and the program runs with your assignment.

▪▪ P2.11 Write a program that asks the user to input

- The number of gallons of gas in the tank
- The fuel efficiency in miles per gallon
- The price of gas per gallon

Then print the cost per 100 miles and how far the car can go with the gas in the tank.

▪ P2.12 *File names and extensions.* Write a program that prompts the user for the drive letter (`C`), the path (`\Windows\System`), the file name (`Readme`), and the extension (`txt`). Then print the complete file name `C:\Windows\System\Readme.txt`. (If you use UNIX or a Macintosh, skip the drive name and use `/` instead of \ to separate directories.)

▪▪ P2.5 Enhance the output of Exercise P2.4 so that the numbers are properly aligned:

```
Sum:          45
Difference:   -5
Product:     500
Average:      22.50
Distance:      5
Maximum:      25
Minimum:      20
```

▪▪▪ P3.28 *Roman numbers.* Write a program that converts a positive integer into the Roman number system. The Roman number system has digits

I	1
V	5
X	10
L	50
C	100
D	500
M	1,000

L 50
XL 40
LX 60

Numbers are formed according to the following rules:

a. Only numbers up to 3,999 are represented.

b. As in the decimal system, the thousands, hundreds, tens, and ones are expressed separately.

c. The numbers 1 to 9 are expressed as

I	1
II	2
III	3
IV	4
V	5
VI	6
VII	7
VIII	8
IX	9

© Straitshooter/iStockphoto.

V
VI
VII

As you can see, an I preceding a V or X is subtracted from the value, and you can never have more than three I's in a row.

d. Tens and hundreds are done the same way, except that the letters X, L, C and C, D, M are used instead of I, V, X, respectively.

Your program should take an input, such as 1978, and convert it to Roman numerals, MCMLXXVIII.

▪▪ P3.29 Write a program that asks the user to enter a month (1 for January, 2 for February, and so on) and then prints the number of days in the month. For February, print "28 or 29 days".

```
Enter a month: 5
30 days
```

Do not use a separate `if/else` branch for each month. Use Boolean operators.

▪▪▪ P3.30 French country names are feminine when they end with the letter e, masculine otherwise, except for the following which are masculine even though they end with e:

- le Belize
- le Cambodge
- le Mexique
- le Mozambique
- le Zaïre
- le Zimbabwe

Write a program that reads the French name of a country and adds the article: le for masculine or la for feminine, such as le Canada or la Belgique.

However, if the country name starts with a vowel, use l'; for example, l'Afghanistan.

For the following plural country names, use les:

- les Etats-Unis
- les Pays-Bas

- **"split()"**: The split splits a string into a list. You can specify the separator, default separator is any whitespace.

sent = "Use Boolean operators"
li = sent.split()

0 1 2
li = ["Use", "Boolean", "operators"]
-3 -2 -1

liAnother = sent.split("e")

li = ["Us", " Boole", "an Op", "rators"

```
>>> sent
'Use Boolean Operators'
>>> sent.split()
['Use', 'Boolean', 'Operators']
>>> li = sent.split()
>>> li
['Use', 'Boolean', 'Operators']
>>> for ele in li:
...     len(ele)
...
3
7
9
```

split(separator, maxsplit)

#separator: Optional. specifies the separator to use when splitting the string. By default, any white space is a separator.

#maxsplit: Optional specifies how many splits to do. default value is minus one, which is "all occurrences".

```
>>> liAnother = sent.split("e")
>>> liAnother
['Us', ' Bool', 'an Op', 'rators']

>>> liAnother = sent.split("e")
>>> liAnother
['Us', ' Bool', 'an Op', 'rators']
>>> text = "boolean#int#str#list#dict"
>>> li1 = text.split("#", 1)
>>> li1
['boolean', 'int#str#list#dict']
>>> li2 = text.split("#", 2)
>>> li2
['boolean', 'int', 'str#list#dict']
>>> li3 = text.split("#", 3)
>>> li3
['boolean', 'int', 'str', 'list#dict']
>>>
```

"partition()":

```
>>> txt = "I could eat Cherry all day"
>>> x = txt.partition("Cherry")
>>> x
('I could eat ', 'Cherry', ' all day')
>>>
```

The partition() method searches for a specified string, and splits the string into a tuple containing three elements.
The first element contains the part before the specified string.
The second element contains the specified string.
The third element contains the part after the string.

Note: This method searches for the first occurrence of the specified string.

string.partition(value):
value: Required. The string to search for.

Q. WAP to input a sentence and find out the longest and shortest word from the sentence. For example:
Input:
Enter any sentence: I could eat Cherry all day
Output:
Longest word: Cherry
Shortest word: I

Q. WAP to input a sentence and calculate how many vowels, consonants, and spaces are present.
Enter any sentence: I could eat Cherries all day
Output:
Number of vowels: 10
Number of consonants: 13
Number of spaces:5

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt ⇒	I		c	o	u	l	d		e	a	t		c	h	e	r	r	i	e

	19	20	21	22	23	24	25	26	27
txt ⇒	s		a	l	l		d	a	y

```
vowels = "AEIOUaeiou"
countConsonant = 0
countVowels = 0
countSpace = 0
i = 0
while i < len(txt):

    c = txt[i]
    if c.isspace():
        countSpace += 1
    elif c.isalpha():
        if vowels.find(c) > -1:
            countVowels += 1
        else
            countConsonant += 1
    i = i + 1
print("Number of vo....)
```

Homework:
1. Write a program to accept two different characters and display the

sum and difference of their ASCII values.
Sample Input: A
d
Sample Output: The sum of ASCII values = 165
The difference of ASCII value = 35

2. Write a program to input an alphabet in upper case or in lower case. Display the next alphabet accordingly. (i.e.: 'a' follows 'b',.. 'z follows 'a')

3. Write a program to accept a character. If it is a letter, then display the case (i.e., lower or upper case), otherwise check whether it is a digit or a special character.
Sample Input: p
Sample Output: p is a letter
p is in lower case.

4. Write a program to accept a word or a String and display the new string after removing all the vowels present in it.
Sample Input: COMPUTER APPLICATIONS
Sample Output: CMPTR PPLCTNS

5. Write a program to accept a word or a string. Count all the alphabets excluding vowels present in the word/string and display the result.
Sample Input: Happy New Year
Sample Output: No. of alphabets excluding vowels = 8

6. Write a program to accept any name and display only the initials(i.e., first alphabet of each word).
Sample Input: Avul Pakir Jainulabdeen Abdul Kalam
Sample Output: A.P.J.A.K

' ' ← symbol of space bar

```
str1 = input('Enter any sentence: ')
vowels = "AEIOUaeiou"
for char in str1:
    if char not in vowels:
        print(char, end="")
```

variable

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
str1 →	C	O	M	P	U	T	E	R	_	A	P	P	L	I	C	A	T	I	O	N

char = 'C' 'O' 'M' 'P' 'U' 'T' 'E' ' ' 'A' 'P'

vowels = A E I O U a e i o u

output: CMPT

Or

```
i = 0
length = len(str1)
while i < length:
    char = str1[i]
    if char not in vowels:
        print(char, end="")
    i += 1
```

7. Write a program in Python to accept a name containing three words and display the surname first followed by the first and middle names.
Sample Input : MOHANDAS KARAMCHAND GANDHI
Sample output : GANDHI MOHANDAS KARAMCHAND

8. Write a program in Python to enter a String/Sentence and display the longest word and the length of the longest word present in the String.

Sample Input "TATA FOOTBALL ACADEMY WILL PLAY AGAINST MOHAN BAGAN"
Sample output : The longest word FOOTBALL The length of the word:8

9. Write a program in Python to accept a String and find the frequency of each vowel present in the String.
Sample Input "RAIN WATER HARVESTING ORGANISED BY JUSCO"
Sample output :
   Frequency of 'A' or 'a' = 4
   Frequency of 'E' or 'e' = 3
   Frequency of 'I' or 'i' = 3
   Frequency of 'O' or 'o' = 2
   Frequency of 'U' or 'u' = 1

10. Write a program in Python to accept a String as: "IF IT RAINS, YOU WILL NOT GO TO PLAY" Convert all characters of the words in lower case other than the first characters so as to obtain the following output: Sample output: If It Rains, You Will Not Go To Play

Home work(1-02-2023)

11. Write a program in Python to accept a String and print Unicode value of each character in the string:
Sample Input: PYTHON PROGRAMMING
Output:
   The Unicode values of the characters are:
   P = 80
   Y = 89
   T = 84
   H = 72
   O = 79
   N = 78
   Space = 32
   P = 80
   R = 82
   O = 79
   G = 71
   R = 82
   A = 65
   M = 77
   M = 77
   I = 73
   N = 78
   G = 71

12. Write a program in Python to accept a String and replace all the vowels with Asterisk (*) present in the String.
Sample Input "TATA STEEL IS IN JAMSHEDPUR"
Sample output: T*T* ST**L *S *N J*MSH*DP*R

13. Write a program in Python to enter a String and frame a word by joining all the first character of each word. Display the new word.
Sample Input :Vital Information Resource Under Seize
Sample Output: VIRUS

14. Write a program in Python to enter a String and display all the palindrome words present in the String.
Sample Input :MOM AND DAD ARE NOT AT HOME
Sample Output: MOM
   DAD